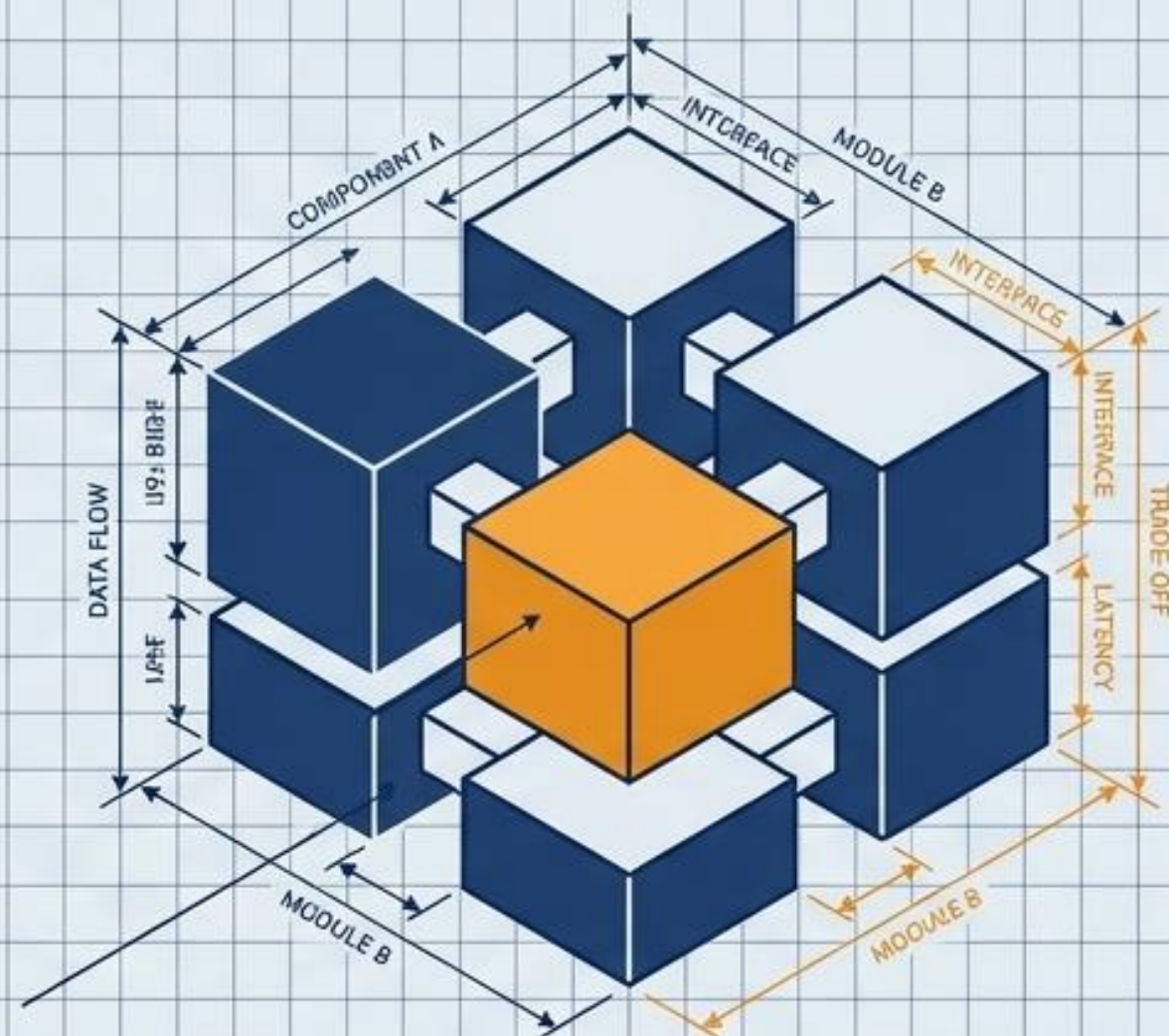
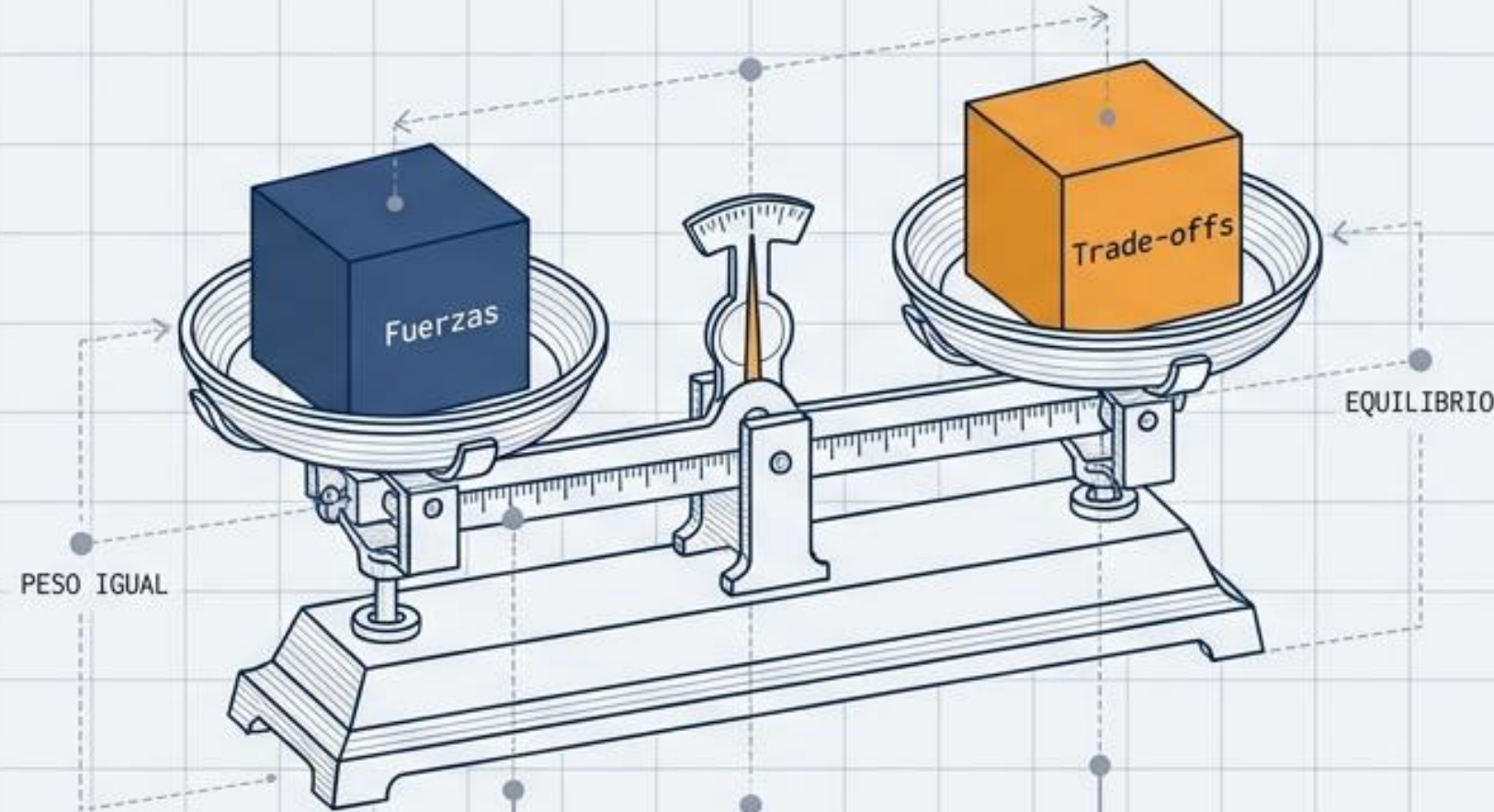




El Criterio Arquitectónico

Patrones de Diseño, Estructuras Ligeras
y el Arte de los Trade-offs

No Existen Soluciones Perfectas, Solo Decisiones Intencionales



CONTEXTO

EQUILIBRIO DE FUERZAS

DECISIÓN INTENCIONAL

El Paradigma

Un patrón de diseño no es una regla inquebrantable. Es una solución estándar a un problema recurrente dentro de un contexto específico.

Patrón vs. Anti-patrón

Aplicar un patrón fuera de su contexto ideal, o sin comprender qué fuerzas intenta equilibrar, lo convierte automáticamente en un anti-patrón.

La Regla de Oro

La madurez técnica de un arquitecto radica en saber exactamente cuándo NO usar un patrón.

El Ecosistema de los Patrones: Vista de Alto Nivel

Creacionales



Qué Resuelven

Determinan cómo se crean los objetos sin acoplar el sistema a implementaciones concretas.

Fuerza Principal



Flexibilidad vs Simplicidad

Riesgo

Exceso de abstracción.

Estructurales



Qué Resuelven

Organizan cómo se relacionan los componentes para formar estructuras más grandes.

Fuerza Principal



Reutilización vs Complejidad

Riesgo

Diseño innecesariamente complejo.

Comportamiento



Qué Resuelven

Definen cómo interactúan los objetos y cómo se delegan las responsabilidades.

Fuerza Principal



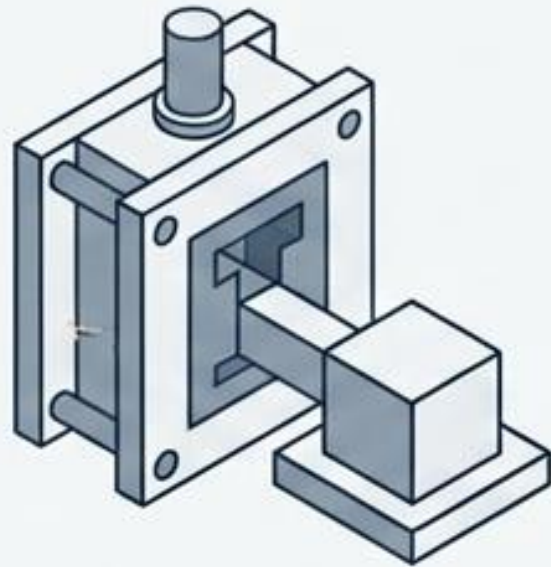
Desacoplamiento vs Trazabilidad

Riesgo

Difícil seguimiento del flujo de la aplicación.

Patrones Creacionales: El Nacimiento del Objeto (Parte I)

Factory Method



Problema

Evitar acoplarse a clases concretas.



Solución

Delegar la creación a subclases.

Fuerza



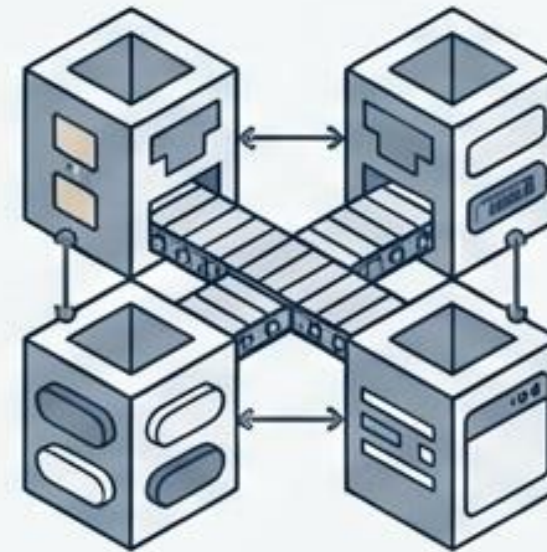
Extensión

Simplicidad

Trade-off

El sistema queda abierto a extensión, pero requiere crear muchas más clases.

Abstract Factory



Problema

Crear familias de objetos relacionados garantizando consistencia.



Solución

Agrupar la creación de productos bajo una única interfaz abstracta.

Fuerza



Consistencia

Flexibilidad individual

Trade-off

Fuerte cohesión entre productos, pero es muy difícil agregar nuevos tipos a la familia.

Patrones Creacionales: Construcción y Precaución

Builder



Problema

Construcción compleja paso a paso (ej: armar un objeto con 10 parámetros opcionales).



Solución

Extraer el código de construcción a objetos "constructores" independientes.

Fuerza



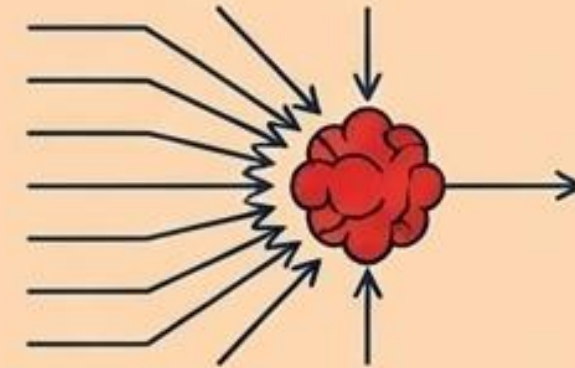
Claridad

Verbosidad

Trade-off

El código es mucho más largo de escribir, pero inmensamente más legible y seguro al instanciar objetos complejos.

Advertencia Arquitectónica: Singleton



Problema Original:

Garantizar una única instancia (ej. config, logger).
Fuerza: Control global vs Testabilidad.

El Problema Arquitectónico:

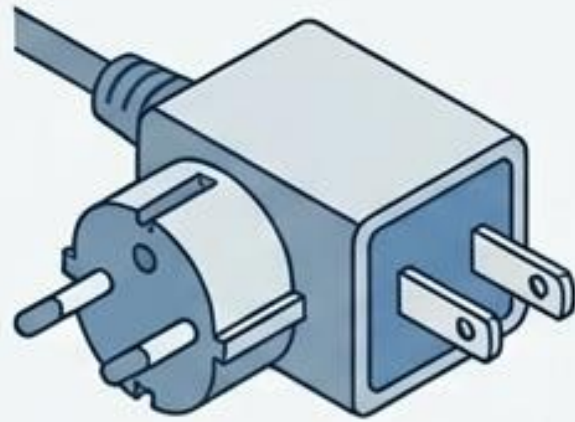
- Genera acoplamiento oculto en todo el sistema.
- Dificulta críticamente el testing automatizado.
- Puede convertirse rápidamente en un estado global tóxico.

Veredicto Moderno:

En arquitecturas modernas, se debe evitar.
Se prefiere resolver la unicidad mediante la Inyección de Dependencias.

Patrones Estructurales: El Esqueleto (Parte I)

Adapter



Problema

Interfaces incompatibles que necesitan comunicarse obligatoriamente.



Solución

Envolver la interfaz incompatible con una clase adaptadora sin alterar el código original.

Fuerza



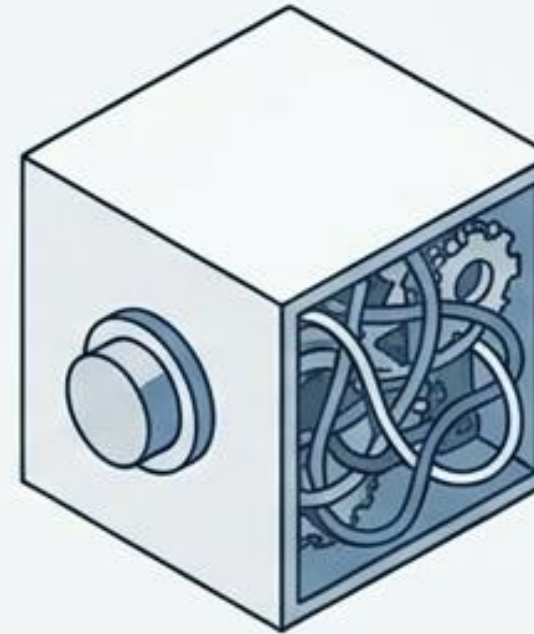
Reutilización

Pureza de diseño

Trade-off

Permite reutilizar código legacy, pero introduce una capa extra de indirección y complejidad al esqueleto.

Facade



Problema

Múltiples subsistemas demasiado complejos para ser operados por el cliente.



Solución

Proveer una interfaz unificada, de alto nivel y altamente simplificada.

Fuerza



Simplicidad

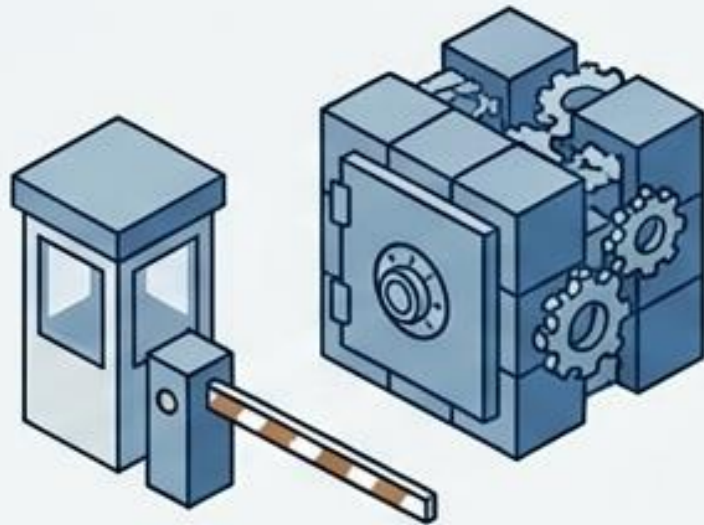
Control fino

Trade-off

Oculto el ruido y es vital en la arquitectura por capas, pero puede limitar a usuarios avanzados que requieren funcionalidades específicas del subsistema.

Patrones Estructurales: Control y Composición Dinámica

Proxy



Problema: Controlar el acceso a un objeto costoso o remoto.

Tipos de Uso: Lazy loading, Seguridad, Proxy Remoto.

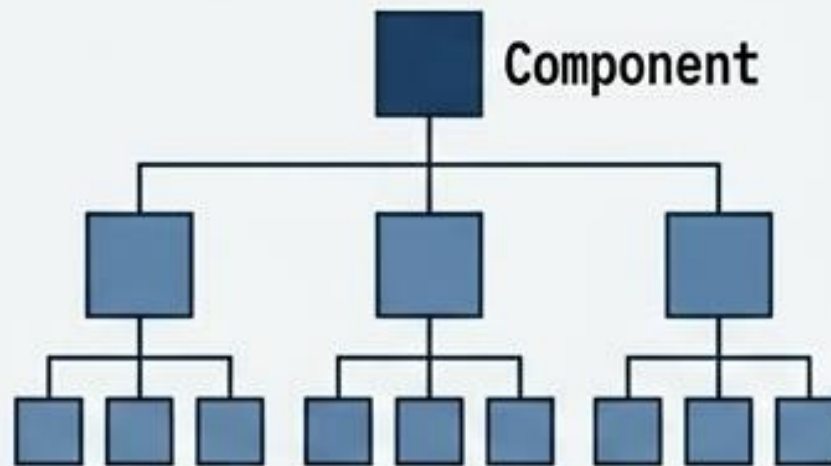
Fuerza



Trade-off:

Añade seguridad y optimización, pero retrasa la ejecución inicial.

Composite



Problema:

Tratar objetos individuales y compuestos bajo la misma interfaz (ej: Árbol de archivos).

Fuerza



Trade-off:

Simplifica dramáticamente el código del cliente, pero sobre-generaliza el diseño del sistema.

Decorator



Problema: Agregar comportamiento y responsabilidades a un objeto de forma dinámica en tiempo de ejecución.

Fuerza



Trade-off:

Máxima flexibilidad, pero genera sistemas difíciles de depurar visualmente debido a las múltiples capas.

Patrones de Comportamiento: La Coreografía (Parte I)

Strategy



🔒 Problema

Variar un algoritmo o lógica de negocio en tiempo de ejecución de manera limpia.

🔑 Solución

Encapsular cada algoritmo en una clase separada bajo una interfaz común.

Fuerza



Flexibilidad

Cantidad de clases

Trade-off

Elimina los condicionales gigantes, pero arriesga una proliferación masiva de clases si el algoritmo rara vez cambia.

Observer



🔒 Problema

Dependencias de estado uno-a-muchos (Manejo de Eventos, Reactividad).

🔑 Solución

Mecanismo de suscripción para notificar a múltiples objetos sobre cambios de estado.

Fuerza



Desacoplamiento

Riesgo de cascadas

Nota Arquitectónica

Es la base conceptual para la moderna Event-driven architecture. Su mayor riesgo es desencadenar cascadas de eventos incontrolables difíciles de depurar.

Patrones de Comportamiento: Flujo y Estado (Parte II)

Command



Problema: Encapsular acciones como objetos.

- Permite: Funcionalidad Undo, Logging de operaciones, y encolamiento de tareas (Colas).

Fuerza: Flexibilidad máxima **vs** Overhead de creación masiva de objetos de comando.

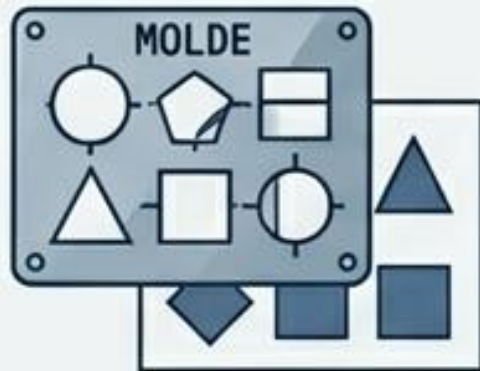
State



Problema: Comportamiento de un objeto que depende estrictamente de su estado actual.

- Evita la creación de bloques if/else o switch gigantes.

Fuerza: Claridad lógica absoluta **vs** Aumento dramático en la cantidad de clases.



Template Method

Algoritmo base fijo con pasos específicos variables.

Fuerza: Reutilización **vs** Rigidez estructural.



Iterator

Recorrer colecciones sin exponer su estructura interna.

Fuerza: Encapsulamiento **vs** Capa extra.

Como se relacionan entre ellos?

Creacionales - Relaciones con otros Patrones

Factory Method → Builder

Muchos diseños empiezan con Factory Method (menos complicado) y evolucionan hacia Abstract Factory, Prototype o Builder (más flexibles, pero más complicados).

Builder vs Abstract Factory

Builder construye objetos complejos paso a paso. Abstract Factory crea familias de objetos relacionados y devuelve el producto inmediatamente. Builder permite pasos adicionales antes de extraer el producto.

Builder + Composite

Puedes utilizar Builder al crear árboles Composite complejos, programando sus pasos de construcción para que funcionen de forma recursiva.

Builder como Singleton

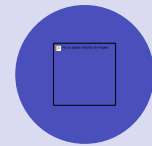
Los patrones Abstract Factory, Builder y Prototype pueden todos ellos implementarse como Singletons.

Comportamiento - Relaciones con otros patrones



Visitor + Command

Puedes tratar a Visitor como una versión potente del patrón Command. Sus pueden ejecutar operaciones sobre varios



Visitor + Composite

Puedes utilizar el patrón Visitor para ejecutar una operación sobre un árbol Composite entero.



Visitor + Iterator

Puedes utilizar Visitor junto con Iterator para recorrer una estructura de datos compleja y ejecutar alguna operación sobre sus elementos, incluso aunque todos tengan clases distintas.

Comportamiento - Relaciones con otros Patrones

State, Strategy y Adapter

Tienen estructuras muy similares. Todos se basan en la **composición**, que consiste en delegar trabajo a otros objetos. Sin embargo, todos solucionan problemas diferentes. Un patrón no es simplemente una receta para estructurar tu código, también puede comunicar a otros desarrolladores el problema que resuelve.

State como extensión de Strategy

Ambos patrones se basan en la composición: cambian el comportamiento del contexto delegando parte del trabajo a objetos objetos ayudantes. **Strategy** hace que estos objetos sean completamente independientes y no se conozcan entre sí. Sin embargo, **State** no restringe las dependencias entre estados concretos, permitiéndoles alterar el estado del contexto a voluntad.

Estructurales -Relaciones con otros Patrones

Adapter vs Proxy vs Decorator

Con **Adapter** se accede a un objeto existente a través de una interfaz diferente. Con **Proxy**, la interfaz sigue siendo la misma. Con **Decorator** se accede al objeto a través de una interfaz mejorada.

Facade vs Proxy

Facade es similar a Proxy en que ambos pueden almacenar temporalmente una entidad compleja e inicializarla por su cuenta. Al contrario que Facade, **Proxy tiene la misma interfaz** que su objeto de servicio, lo que hace que sean intercambiables.

Decorator vs Proxy

Ambos tienen estructuras similares y se basan en el **principio de composición**. La diferencia: un Proxy gestiona el ciclo de vida de su objeto de servicio por su cuenta, mientras que la composición de los **Decoradores siempre está controlada por el cliente**.